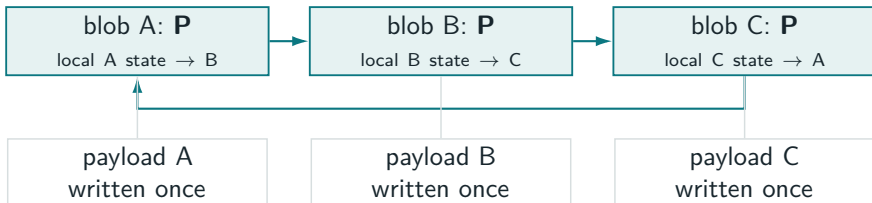


Crash-atomic append publication, built on ordinary blobs

Participant-linked UNO / V2 with write-once payload and checked pages

Status: opt-in ALPHA API; ordinary Storage and V0/V1 remain unchanged.



concurrent inode barriers closed recovery ring no coordinator file

The contract we actually need



FAULT MODEL

Any subset of writes not covered by a completed durability barrier may survive.

INDETERMINATE WINDOW

A failed or cancelled sync may recover old or complete new—never a mixture.

PERFORMANCE TARGET

Keep ordinary append behavior: write payload once and pay at the existing sync.

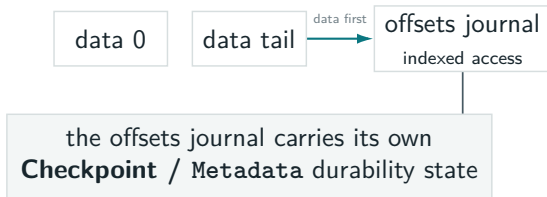
Atomicity is a crash-recovery property; durability remains attached to sync or completion of `start_sync`.

Before: each journal coordinated its own crash state

LEGACY FIXED JOURNAL



LEGACY VARIABLE JOURNAL

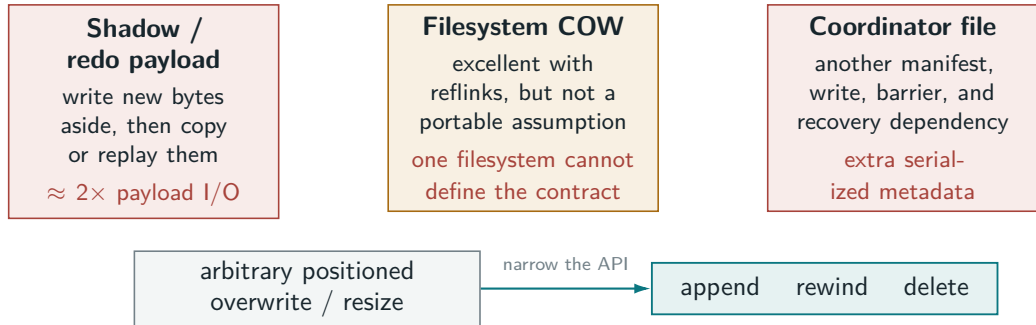


- commit, start_sync, and stronger sync kept data, indexes, and recovery hints in a safe order.
- The storage API had no primitive for atomically publishing several blob roots together.

The journals were correct, but every compound structure owned a bespoke durability protocol.

The append-only boundary makes rollback cheap

A GENERAL TRANSACTION ENGINE WOULD NEED ROLLBACK MATERIAL



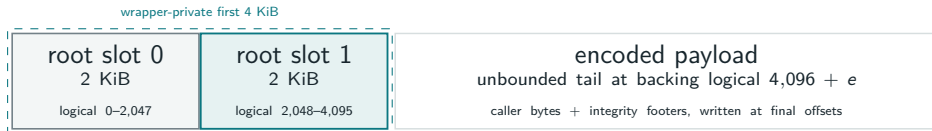
A checksum can detect a torn overwrite; it cannot restore overwritten bytes. Append-only preserves the old prefix, so a root can select a complete visible prefix.

The decisive optimization is an API constraint: preserve old bytes and publish a new length.

Architecture: UNO wraps an ordinary Blob



THE WRAPPER'S BACKING-BLOB COORDINATES



PROTOCOL OWNS

Payload, roots, reads, rewinds, resize, sync, and start_sync all pass through Blob. The publisher adds exact-name and linked-decision recovery.

FILESYSTEM ENVELOPE

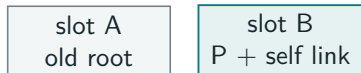
Physical 0-4 KiB is the immutable CWIL kind/version/incarnation envelope for safe name recovery. The backing view begins there: roots are physical 4-8 KiB; payload starts at 8 KiB.

UNO is a protocol adapter over ordinary blob I/O, not a second storage backend.

One blob: the durable decision fits in one barrier



covers earlier payload + root + witness
eligible bounded suffix; otherwise payload preflushes first



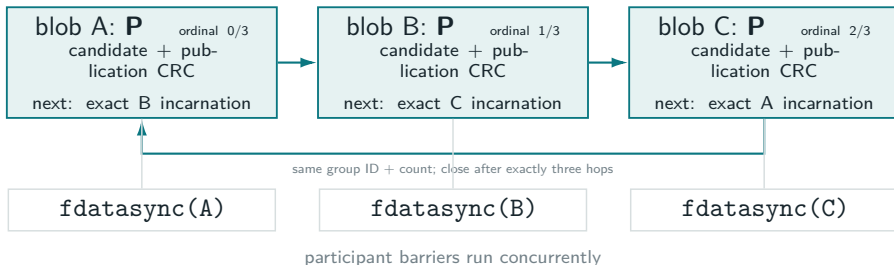
the old root is never overwritten
while publishing its successor

ON RESTART

- If the prepared root, self link, and publication-suffix CRC all validate: expose new.
- Otherwise: expose the preceding committed root.
- No follow-up $P \rightarrow C$ marker and no second durability barrier.

The self-link is the durable decision; later compatible epochs alternate and supersede it.

Several blobs: each root stores one link in a closed ring



WHAT ONE LOCAL WITNESS CARRIES

Fresh 128-bit group ID, count + ordinal, local incarnation/candidate/removal bit, bounded publication-suffix CRC32C, and only the successor's partition/name/incarnation.

Live process: one typed Decision in RAM. On disk: distinct local links only.

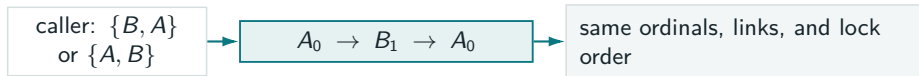
WHAT RESTART PROVES

Opening one unresolved P follows exactly the declared ring, growing state only after each valid link. Only one unique, consecutive, closed ring can publish the complete new group.

No coordinator file and no replicated group descriptor: the participant files are the recovery index.

Overlapping atomic groups

EVERY RING SORTS PARTICIPANT KEYS: (PARTITION, RAW BLOB NAME)



EXACT PARTICIPANT SET REPEATS

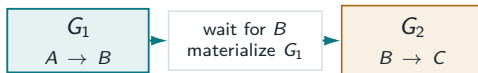


Adjacent generations supersede directly; no $P \rightarrow M$ rewrite.

Shared participant

serialize that blob: G_2 derives from durable G_1 .

PARTICIPANT SET CHANGES



Shared- B serialization is required; eager materialization is current policy.

Disjoint participants

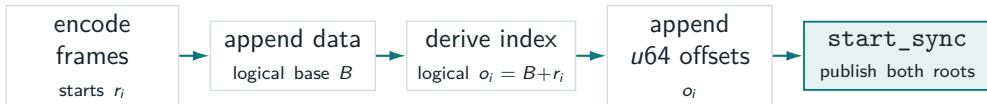
protocol: no shared order;
no temporal dependency.

Lexicographic links and shared-blob serialization prevent cycles without imposing a storage-wide order.

Dependent writes can be built, then published together

VARIABLE JOURNAL APPEND: SEQUENTIAL STAGING IS ALLOWED

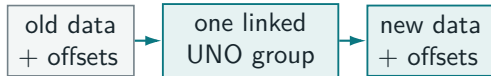
build phase: pending writes may depend on earlier results



HOW VALUE / IS LOCATED

```
start = offsets[i]
end = offsets[i + 1]
decode data[start..end]
 $B$ ,  $r_i$ ,  $o_i$ , start, and end are AtomicWriter logical
coordinates: CRC footers are excluded.
```

WHAT BECOMES VISIBLE

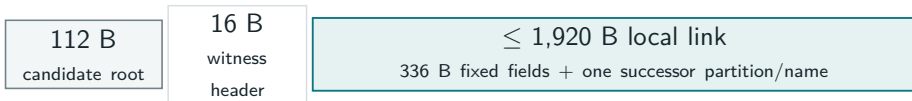


Restart sees the old pair or complete new pair—never mismatched data and offsets lengths.

Stage data, derive footer-free logical offsets, then publish both checked-page roots in one group.

Batch scale: fixed local metadata per participant

EVERY DIRTY PARTICIPANT: ONE 2 KIB SLOT CONTAINS ITS ROOT AND LOCAL WITNESS



PARTICIPANT METADATA



Each participant names only its next exact incarnation. The paths are distributed once around the ring rather than gathered into one record.

PAYLOAD SIZE: NO PROTOCOL BATCH CAP

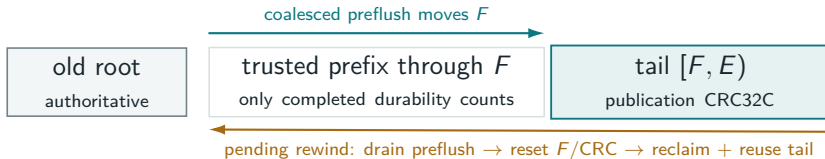


At most **64 MiB aggregate** remains for publication recovery CRC work. Older immutable bytes preflush while appends continue.

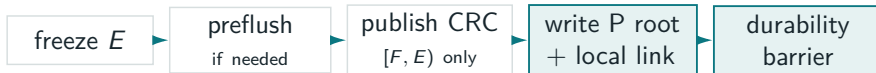
Metadata grows one local link per participant; payload is streamed once rather than copied.

Large epochs bound publication recovery work

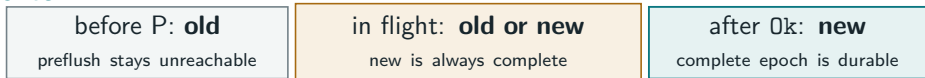
WHILE APPENDING: PAYLOAD IS AT FINAL OFFSETS; NO CANDIDATE ROOT EXISTS YET



AT SYNC: FREEZE ONE EPOCH, THEN PUBLISH



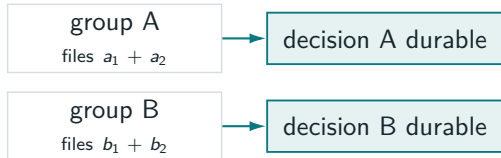
CRASH OUTCOME



Repeat per participant: no payload-byte cap, at most 64 MiB aggregate restart tail, and concurrent final barriers.

Independent groups have no temporal dependency

INDEPENDENT GROUPS



No protocol-wide order

no root WAL, coordinator,
LSN, or commit sequence

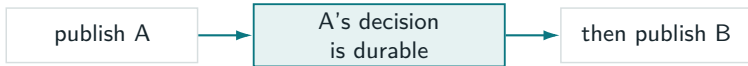
What this avoids

no global transaction lock,
WAL, or volume layer

CRASH STATES FOR INDEPENDENTLY STARTED A AND B

old A / old B | new A / old B | **old A / new B is possible** | new A / new B

WHEN B HAS A TEMPORAL DEPENDENCY ON A



old/old, new/old, new/new
never old A / new B

Independent groups are independent; encode a real dependency in one group or wait for the earlier durable edge.

Boundary: Freezer materializations stay ordinary

IMMUTABLE ARCHIVE IS COMPOSITE: ONLY AUTHORITY NEEDS APPEND-ONLY ATOMIC PUBLICATION

NOT CONVERTED IN PLACE



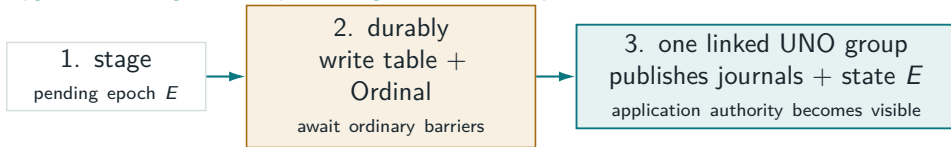
These tables overwrite slots. Their local CRCs detect damage but cannot roll bytes back.

APPEND-ONLY PIECES CAN PARTICIPATE



Journals and an archive-state authority can move together.

LOGICAL COMMIT WITH ONE EXPLICIT TEMPORAL DEPENDENCY



before UNO: **old** | in flight: **old or complete new** | after UNO: **new**

If old wins, future materializations remain unreachable; if new wins, their durability was already proven.

Ordinary materializations may be written and made durable before UNO publishes the authority that reaches them.

Recovery is correct for arbitrary surviving write subsets

P = prepared linked candidate M = materialized retained root T = durable tombstone

What validates after restart	What recovery can prove	Exposed result
No M/T proof; every exact P, local link, incarnation, candidate, and publication suffix validates	One group ID/count; unique consecutive ordinals close after exactly N hops	New roots for the whole group
No M/T proof; any P, link, incarnation, candidate, or publication suffix is missing/mismatched	The linked group decision is incomplete	Old roots for the whole group
An exact materialized M or tombstone T validates	Installation began only after the P ring was durable	Finish peers; after unlink begins, resume the descending frontier from ordinal 0

TORN ROOTS

Root and witness-envelope checksums reject mixed spellings and partial installation. P is not a final standalone root.

TORN PAYLOAD

A trusted prefix plus publication CRC32C rejects partial visibility with probabilistic checksum assurance.

THREAT BOUNDARY

CRC32C detects crash damage on trusted local disk; it is not tamper authentication.

The protocol never assumes that surviving writes form a prefix in submission order.

Append is the fast path; delete is ordered by phase

APPEND

write final tail once
publish longer root
no relocation or compaction

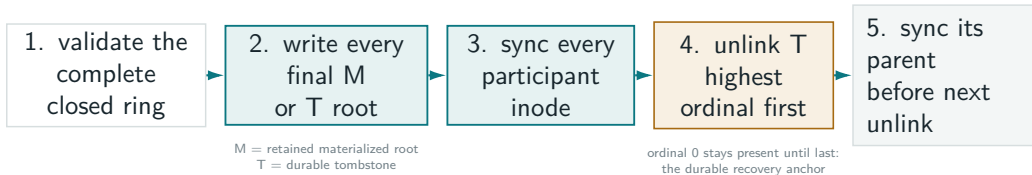
REWIND

publish shorter root
then `Blob::resize`
restart repeats a lost truncate

DELETE

durable tombstone
then unlink + directory sync
no separate delete manifest

GROUP DELETE: FINISH THE RING WALK BEFORE CHANGING THE NAMESPACE

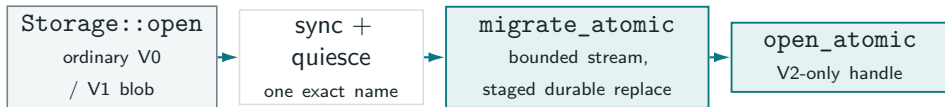


Rewind keeps the old root as rollback until the shorter root is durable. After delete step 3, each M is independent; descending durable unlinks keep every remaining T reachable from ordinal 0.

All final roots first; then a durable descending unlink frontier. Ordinal 0 replaces a delete manifest.

Migration stays inside initialization

THE STORAGE PRIMITIVE MIGRATES ONE QUIESCED NAME AT A TIME



EXPLICIT AND RESUMABLE

- Ordinary Storage formats do not migrate implicitly; `open_atomic` rejects an ordinary blob.
- The filesystem copy uses bounded memory, synchronizes the staged inode, durably replaces the exact generation, and is safe to retry.

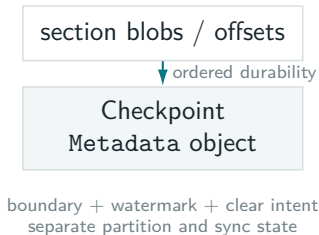
CONTIGUOUS INIT

- Legacy checked pages have different framing, so `init` decodes and re-encodes one section blob at a time into native checked pages.
- Only `init` sees mixed state. A successful call returns fixed or variable application code that uses V2 exclusively.

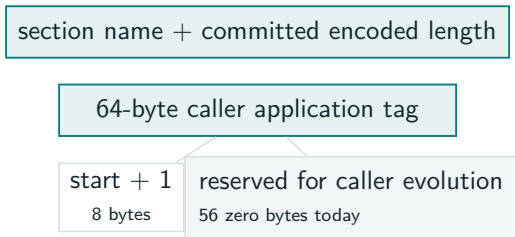
Migration is a bounded init-time bridge, not a compatibility branch in the steady-state journal.

Contiguous V2 drops the Metadata checkpoint

BEFORE



NATIVE V2 STEADY STATE

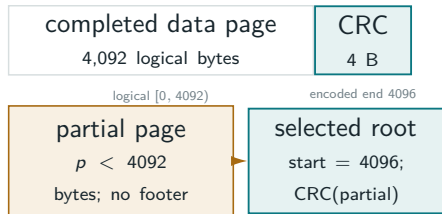


- Both fixed and variable contiguous use `Chunked(4092)` checked pages: 4,092 logical bytes + a V2-owned 4-byte footer.
- Variable V2 stages data first, derives logical offsets, then publishes matching data + offsets roots in the **same UNO group**.
- One fixed section or variable pair holds `start + 1`; page geometry maps encoded lengths to logical bounds.
- `MutableV2`: consuming mutations, `start_sync`, and `sync`—no legacy commit split.

Offsets remain as the variable index; the separate checkpoint object disappears.

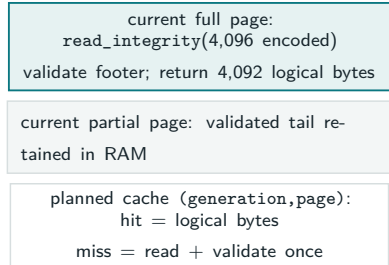
Checked pages: immutable footer, bounded open tail

CHUNKED(4092) ENCODING



sync stores the partial page's start and rolling CRC in the root without closing it. At 4,092 bytes, V2 appends the footer and the 4,096-byte encoded page becomes immutable.

CURRENT AND PLANNED READS



Today cache-only reads miss. The plan coalesces faults; rewind invalidates the shortened page and suffix.

Coordinates: AtomicBlob *e* is encoded and includes footers; AtomicWriter/journal

l is logical and excludes them. Backing = 4096 + *e*; filesystem = 8192 + *e*.

The root protects the open page; every completed page has an immutable footer and is lazily cacheable.

CRC32C has four separate jobs

Root CRC

Protects generation, length, integrity state, and caller tag.

Link-envelope CRC

Protects local witness framing after a prepared root.

Publication CRC

Revalidates only the bounded speculative suffix.

Unit CRC

Footer when complete; root field while open.

THE GENERIC INTEGRITY API CHOOSES UNIT BOUNDARIES; CALLERS STILL OWN INDEXES

Caller-delimited Variable

Continue streams one unit;
Complete adds its footer.
Generic API—**not** contiguous.

Chunked(n)

Closes every n data bytes. Both contiguous forms bind $n = 4092$.

Read / rewind

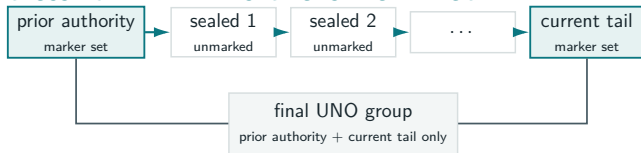
Validate one unit; rewind rebuilds the retained open-unit CRC when needed.

A compare token fences stale integrity writers before payload I/O; raw `Blob::read_at` exposes encoded bytes without unit validation.

CRC32C rejects accidental crash damage probabilistically; it is neither authentication nor rollback material.

Reopen cost is bounded at both layers

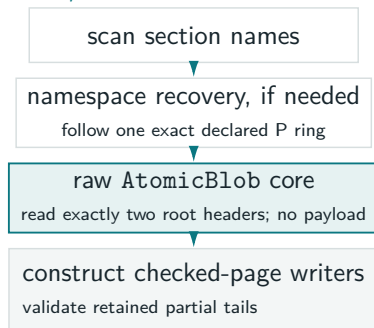
CROSSING MANY FIXED SECTIONS IN ONE EPOCH



Sealed intermediate sections may be durably staged while unreachable; their names fill the interval once the marker publishes.

fixed: **2 roots** variable: **4 roots**
prior/current × data/offsets, all in one group

OPEN / RESTART PATH



Fixed: ≤ 1 tail per retained section. Variable: sealed pairs are complete; active data + offsets: ≤ 2 tails. Full pages remain lazy.

Raw core reads two root slots; higher layers validate only bounded partial tails.

Benchmark method: paired ordinary blobs vs UNO

ONE OPERATION = FOUR INDEPENDENT 4 MIB CONTIGUOUS APPENDS

ordinary side: four
Blob::write_at calls

start four per-file syncs
in parallel; await all

UNO side: four final-
offset payload writes

one distributed root decision;
same four per-file barriers

CONTROLLED WORKLOAD

- 4 blobs; one 4 MiB append per blob per operation
- 100 operations per side and seed; inflight 1
- paired baseline, contiguous shape, bench profile
- Tokio runtime with 2 workers

INTEGRITY MODES

- none: isolates wrapper + linked publication
- chunked, data size 4092: includes page CRCs and footers
- Report UNO / ordinary paired ratio, plus each side's MiB/s

```
cargo bench -p commonware-runtime -bench storage - --workload write_atomic_batch_append -paired-baseline -blobs 4  
-write-shape contiguous -inflight 1 -io-size 4M -appends-per-batch 1 -operations 100 -integrity-mode MODE -seed N -output json
```

The V1-backed wrapper should preserve protocol performance while removing backend duplication; none is the direct check.

Fresh paired results: UNO remains close to ordinary

Filesystem	Mode	UNO / ordinary	Ordinary MiB/s	UNO MiB/s
APFS SSD	raw	0.9783 [0.9245, 1.0617]	711.0 [518.3, 958.5]	685.2 [514.4, 922.5]
APFS SSD	Chunked (4092)	0.9399 [0.9243, 0.9636]	944.9 [624.9, 1089.7]	884.4 [602.2, 1027.6]
ext4 loopback	raw	0.9650 [0.8361, 1.0243]	824.1 [447.0, 1102.9]	844.1 [373.7, 1107.1]
ext4 loopback	Chunked (4092)	0.9241 [0.8971, 1.0152]	936.1 [867.5, 969.9]	880.7 [803.5, 938.1]

MEDIAN [MIN, MAX] ACROSS FIVE PAIRED SEEDS

Bench profile; 100 operations per side and seed; 2 Tokio workers. Each operation appends 4 MiB to each of 4 blobs with inflight 1. Chunked includes CRC32C calculation and one four-byte footer per 4,092 logical bytes.

CRC cost after streaming optimization

Chunked is within 3.8 points of raw on APFS and 4.1 on ext4.

ext4 caveat

Genuine ext4 on /dev/loop0; absolute rates include LinuxKit, loopback, and Docker Desktop effects.

UNO / ordinary: raw 97.8% / 96.5%; checked-page 94.0% / 92.4% (APFS / ext4).

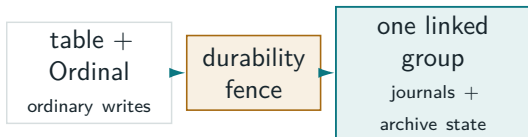
Open questions: indexing and composite authority

SHOULD BLOB OWN MORE INTEGRITY INDEXING?



- Today callers retain completed-unit offsets; raw roots keep only the unfinished unit.
- A shared index could remove duplicate caller machinery and make integrity reads easier.
- Challenge: version and publish the index without extra hot-path barriers or unbounded reopen work.
- Challenge: rewind/reuse needs index truncation, stale-token fencing, and reclamation semantics.

CONVERT COMPOSITE STORES / ARCHIVE AUTHORITY?

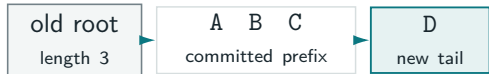


- A full opt-in Archive V2 could publish journal roots plus one archive-state snapshot after ordinary materializations are durable.
- Challenge: versioned migration and a single live authority that need not scan historical state.
- Challenge: republish sealed sections so final group membership remains bounded across rollover.
- Challenge: arbitrary-subset recovery tests spanning tables, Ordinal, journals, pruning, and cleanup.

Keep the primitive narrow until an owner can specify bounded recovery, migration, rewind, and reclamation.

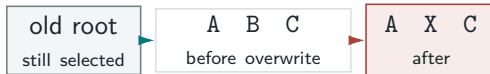
Atomic overwrite is a different storage engine

APPEND-ONLY: THE FALLBACK REMAINS INTACT



The old root still names A B C; selecting it discards D without restoring payload.

OVERWRITE: THE FALLBACK LOSES ITS BYTES



A CRC may reject torn X, but B is gone; the old root cannot restore it.

MAKING THAT OVERWRITE ROLLBACKABLE REQUIRES ANOTHER MECHANISM

COW / patch map

Write replacement elsewhere; publish a page map. Adds indirection and reclamation.

Undo / redo log

Persist old bytes or replay data first. Adds payload writes and another ordering protocol.

Volume layer

Version allocation below every store. Adds shared metadata, locking, and recovery work.

Under the fault model, any subset of unsynced overwrite bytes may survive; rollback therefore needs retained old bytes or a separate new copy.

CRC detects but cannot restore overwrites. Composite V2 keeps overwrite-heavy indexes ordinary behind atomic authority.